

NUTRIAPP

Cahier des charges

Coach nutrition & sport intelligent - application mobile et plateforme web client

Version	1.0
Date	16 septembre 2026
Statut	Cadrage fonctionnel et technique
Clients	Application Flutter + site web Next.js
Backend	Laravel REST + Sanctum + PostgreSQL

Vision : réduire la friction du suivi alimentaire et transformer les données nutritionnelles, le stock du foyer, les préférences, l'activité sportive et le contexte restaurant en recommandations concrètes et personnalisées.

Document de référence pour la conception, le développement, les tests, le déploiement et les évolutions de NutriApp.

1. Vision et objectifs du produit

NutriApp est une application de coaching nutritionnel et sportif intelligent. Elle ne doit pas se limiter à compter les calories : elle doit comprendre la journée de l'utilisateur, simplifier l'enregistrement des aliments et proposer des actions utiles pour la suite.

Le service est accessible depuis **deux interfaces clientes principales** : l'application mobile Flutter et le site web Next.js. Le site web n'est donc pas un simple back-office : il doit proposer aux utilisateurs une expérience fonctionnelle proche de l'application, adaptée à un écran d'ordinateur.

Une interface d'administration pourra être ajoutée ultérieurement dans Next.js, via des routes et autorisations dédiées. Aucune interface administrateur mobile n'est prévue à ce stade.

Objectifs principaux

- Simplifier au maximum la saisie des repas par recherche, favoris, portions, code-barres et, à terme, voix, OCR et photo.
- Suivre calories, protéines, glucides et lipides par repas et par journée.
- Calculer des objectifs nutritionnels personnalisés à partir du profil, de l'activité et de l'objectif.
- Proposer ce qu'il est pertinent de manger ensuite en tenant compte des objectifs restants et des contraintes.
- Gérer le stock du foyer : frigo, congélateur et placards, péremptions et liste de courses.
- Proposer des repas compatibles avec le stock disponible et limiter le gaspillage.
- Gérer plusieurs membres d'un foyer avec un repas commun et des portions adaptées.
- Intégrer progressivement sport, restaurants et automatisations intelligentes.

Principes produit

Principe	Application
Automatisation	Réduire les saisies répétitives et réutiliser les données déjà connues.
Transparence	Distinguer données nutritionnelles officielles, données externes et estimations.
Progressivité	Commencer par des règles métier explicables avant une IA avancée.
Multi-client	Même logique métier et même API pour Flutter et Next.js.
Fiabilité	Les calculs critiques restent côté backend et sont testés.

2. Utilisateurs, interfaces et rôles

Acteur	Interface	Périmètre
Utilisateur	Flutter mobile	Compte, profil, dashboard, aliments, repas, stock, recommandations, famille, sport, restaurants, etc.
Utilisateur	Next.js web	Interface web client similaire à l'application, optimisée pour ordinateur.
Administrateur - futur	Next.js /admin	Gestion et supervision selon les besoins futurs et les droits attribués.
Backend	Laravel	Authentification, autorisations, règles métier, calculs, accès PostgreSQL et intégrations externes.

Parité web / mobile

Les fonctionnalités métier doivent être pensées comme des capacités de l'API et non comme des fonctions propres à un client. Une fonctionnalité disponible sur mobile pourra ainsi être exposée sur le web sans dupliquer la logique métier. Certaines fonctions dépendantes du matériel, comme le scan caméra, pourront avoir une UX différente.

Administration

Le rôle administrateur et la route **/admin** sont prévus pour une phase ultérieure. Le périmètre exact du back-office reste à définir : utilisateurs, référentiels, aliments, restaurants, contenus, signalements, statistiques techniques, etc. Il ne doit pas être confondu avec le site web client.

3. Architecture technique

Couche	Technologies	Responsabilité
Backend	Laravel, API REST, Sanctum	Source de vérité métier, sécurité, validation, calculs, intégrations.
Base	PostgreSQL	Persistance des comptes, profils, aliments, repas, stocks, foyers, etc.
Web client	Next.js, TypeScript, Tailwind CSS	Expérience utilisateur sur ordinateur.
Data web	Axios, TanStack Query	Appels API, cache serveur/client, synchronisation et états de chargement.
Formulaires web	React Hook Form, Zod	Formulaires et validation ergonomique côté client.
Mobile	Flutter	Expérience mobile Android puis iOS.
Data mobile	Dio, Riverpod	HTTP, état applicatif et logique de présentation.
Navigation / secrets	GoRouter, Flutter Secure Storage	Routes mobile et stockage sécurisé des jetons.
Scan	mobile_scanner	Lecture des codes-barres sur mobile.

Organisation du dépôt

```
nutrition-coach/  
|- backend/    Laravel  
|- web/        Next.js  
|- mobile/     Flutter  
|- docs/  
`- infra/
```

Principe d'accès

Flutter et Next.js consomment la même API Laravel et la même base PostgreSQL. Aucun client ne doit accéder directement à PostgreSQL.

4. Authentification et gestion de session

- Inscription utilisateur.
- Connexion email + mot de passe.
- Vérification des identifiants et messages d'erreur compréhensibles.
- Laravel Sanctum pour l'émission et la gestion des jetons.
- Stockage du jeton adapté à chaque client ; Flutter Secure Storage côté mobile.
- Routes protégées et contrôle d'autorisation côté Laravel.
- Déconnexion et invalidation de session/token.
- Vérification d'email et réinitialisation de mot de passe à cadrer/compléter selon l'état du projet.

État actuel

La base d'authentification et les premières pages **Login** et **Register** ont déjà été commencées pour Next.js et Flutter. La priorité est maintenant de poursuivre le site web client.

5. Profil nutritionnel

Chaque utilisateur possède un profil nutritionnel servant aux calculs et recommandations. Les objectifs proposés automatiquement doivent rester modifiables selon les règles produit retenues.

Donnée	Usage
Âge, sexe, taille, poids	Estimation des besoins énergétiques et suivi.
Activité physique	Ajustement de la dépense énergétique estimée.
Objectif	Perte, maintien, prise de poids / masse.
Objectif calorique	Budget énergétique journalier.
Protéines, glucides, lipides	Cibles quotidiennes de macronutriments.
Préférences alimentaires	Personnalisation des suggestions.
Régime / restrictions	Filtrage et contraintes du moteur de recommandations.

Calculs initiaux

Une formule telle que **Mifflin-St Jeor** peut être utilisée pour estimer le métabolisme de base, puis appliquer un facteur d'activité et un ajustement lié à l'objectif. Les coefficients exacts, bornes de sécurité et règles de personnalisation devront être documentés et testés. L'application ne doit pas présenter une estimation comme une mesure clinique.

6. Aliments et base nutritionnelle

La base aliments est centrale. Elle doit combiner un cache local PostgreSQL et des sources externes afin d'éviter des appels inutiles et permettre l'enrichissement progressif des données.

Recherche par EAN

Flux cible : Scan EAN -> recherche PostgreSQL -> si trouvé, retour local -> sinon interrogation Open Food Facts -> normalisation -> enregistrement local -> retour au client.

Données minimales produit

- EAN / code-barres lorsque disponible.
- Nom et marque.
- Calories / 100 g.
- Protéines / 100 g.
- Glucides / 100 g.
- Lipides / 100 g.
- Source et date de récupération des données.
- Informations complémentaires à prévoir : portion, unité, catégorie, allergènes ou autres champs utiles selon les sources.

Modes de saisie futurs

Mode	But	Phase
Recherche / favoris	Saisie rapide des aliments connus.	MVP / proche
Code-barres	Identifier automatiquement un produit emballé.	Proche
OCR étiquette	Extraire les valeurs nutritionnelles d'une étiquette.	Ultérieure
Texte / voix	Comprendre « 2 oeufs et une banane ».	Ultérieure
Photo assiette	Reconnaître / estimer un repas à partir d'une image.	IA avancée

Portions

Pour réduire la dépendance à la balance, NutriApp doit permettre des portions simples : pièce, bol, assiette, cuillère, verre, portion habituelle. Une conversion vers une quantité exploitable doit être définie lorsqu'elle est nécessaire au calcul nutritionnel, avec une indication lorsqu'il s'agit d'une estimation.

7. Repas et suivi journalier

L'utilisateur peut enregistrer petit-déjeuner, déjeuner, dîner et collations. Chaque repas contient plusieurs aliments avec une quantité ou une portion.

Calcul	Résultat attendu
Repas	Calories, protéines, glucides et lipides du repas.
Journée	Somme des repas enregistrés.
Restant	Calories et macros restantes par rapport aux objectifs.
Historique	Consultation des journées précédentes et évolution des données.

Règles importantes

- Les calculs de référence doivent être effectués ou validés côté backend.
- La modification ou suppression d'un aliment d'un repas recalcule les totaux.
- Les unités et portions doivent être normalisées pour éviter des incohérences.
- Les repas pourront être reliés au stock afin de décrétement les quantités consommées.

8. Dashboard utilisateur

Le dashboard est la synthèse quotidienne du coach. Il doit être disponible sur Next.js et Flutter avec une présentation adaptée à chaque écran.

- Calories consommées / objectif / restantes.
- Protéines, glucides et lipides consommés et restants.
- Repas du jour et accès rapide à l'ajout.
- État du stock et produits bientôt périmés.
- Recommandations simples du coach.
- Activité sportive du jour lorsque le module sport sera disponible.

Exemples de recommandations

« Il te reste 600 kcal aujourd'hui. »
« Il te manque environ 35 g de protéines. »
« Voici trois repas adaptés pour ce soir. »
« Ces produits vont bientôt expirer ; tu peux les utiliser dans ce repas. »

9. Stock alimentaire et anti-gaspillage

Le stock représente les aliments disponibles au niveau du foyer. Il couvre le frigo, le congélateur et les placards.

Champ	Description
Produit / aliment	Référence à la base aliments.
Quantité	Quantité restante.
Unité	g, ml, pièce, portion ou autre unité gérée.
Emplacement	Frigo, congélateur, placard ou emplacement futur.
Expiration	Date lorsqu'elle est connue / pertinente.
Foyer	Stock partagé entre les membres autorisés.

Automatisations prévues

- Détecter les produits bientôt périmés.
- Prioriser ces produits dans les recommandations de repas.
- Décrémenter le stock lors de la consommation d'un repas, avec mécanisme de correction.
- Identifier les manques et alimenter la liste de courses.
- Réduire le gaspillage et permettre des statistiques futures.

10. Coach nutritionnel et moteur de recommandations

La première version du coach doit utiliser des **règles métier explicites**. Une IA avancée ne doit pas être nécessaire au fonctionnement du coeur produit.

Entrées du moteur

- Calories et macros restantes.
- Préférences et régime alimentaire.
- Restrictions éventuelles.
- Stock disponible et dates d'expiration.
- Repas déjà consommés.
- Contexte familial lorsque le mode famille est actif.
- Activité sportive lorsque le module est disponible.

Sorties

- Alertes et constats sur la journée.
- Suggestions de repas.
- Priorisation d'aliments à utiliser.
- Ajustement des portions.

- Suggestions de courses ou de préparation.

Traçabilité

Une recommandation doit pouvoir expliquer les principaux facteurs qui l'ont déclenchée. Les estimations doivent être clairement distinguées des données nutritionnelles connues.

11. Plans alimentaires et régimes

Évolutions prévues : génération d'un repas, d'une journée, d'une semaine, planning repas et liste de courses automatique.

Régimes à prendre en charge progressivement

- Méditerranéen
- DASH
- Flexitarien
- Low carb
- Keto
- Jeûne intermittent
- Végétarien
- Végan
- Sans gluten
- Sans lactose
- Montignac

Ces régimes doivent être modélisés sous forme de **règles et contraintes** exploitables par le moteur de recommandations, plutôt que comme de simples étiquettes.

12. Mode famille

Un foyer peut regrouper plusieurs membres. Chaque membre conserve son profil, ses besoins, son objectif et ses restrictions, tandis que certaines ressources sont partagées.

Individuel	Partagé
Profil nutritionnel	Stock / frigo
Objectifs calories et macros	Repas planifiés
Régime et restrictions	Liste de courses
Historique personnel selon règles de confidentialité	Organisation du foyer

Repas commun

Le moteur doit chercher un même repas compatible avec la famille, puis adapter les portions. Exemple : Poulet + riz + légumes - Papa 750 kcal, Maman 550 kcal, Enfant 400 kcal. Les besoins des enfants et situations particulières devront faire l'objet de règles adaptées avant mise en production.

13. Sport

Le module sport est une extension du coach et doit pouvoir utiliser : objectif, niveau, temps disponible et matériel disponible.

- Génération ou sélection de séances.
- Suivi des activités.
- Adaptation cohérente de l'alimentation selon l'activité selon les règles retenues.
- Recommandations pré / post entraînement.

14. Restaurants sains

NutriApp doit aider l'utilisateur lorsqu'il mange à l'extérieur. Une carte permet de découvrir des restaurants et leurs plats.

Objet	Données
Restaurant	Localisation, type de cuisine, informations pratiques et menu.
Plat	Nom, description, calories et macros si disponibles.
Nutrition	Valeurs officielles lorsqu'elles existent ; sinon estimation clairement signalée.
Action utilisateur	Ajouter directement un plat au repas du jour.
Coach	Comparer les plats au budget calorique/macros restant et aux contraintes.

Exemple

« Il te reste 650 kcal. Voici des plats autour de toi compatibles avec ton objectif. »

15. Liste de courses, planning et notifications

- Liste de courses manuelle et générée à partir des besoins.
- Partage de la liste au niveau du foyer.
- Planning alimentaire par repas, journée et semaine.
- Notifications de péremption, rappels et informations utiles configurables.
- Éviter les notifications excessives et permettre le contrôle par l'utilisateur.

16. Pages et navigation web

Le site Next.js est une **interface client complète**. Les pages ci-dessous sont prévues pour les utilisateurs, sauf **/admin** qui correspond à un futur espace d'administration.

Phase	Routes principales
Priorité	/login, /register, /dashboard, /profile, /meals, /foods, /stocks, /history
Ensuite	/recommendations, /restaurants, /family, /shopping-list, /planner, /stats, /settings
Plus tard	/diets, /workouts, /notifications
Administration future	/admin - protégée par rôle et distincte du parcours client

Navigation mobile

Flutter doit couvrir progressivement les mêmes domaines métier. La navigation exacte peut différer afin de rester adaptée au mobile. À ce stade, **aucun back-office administrateur Flutter n'est prévu**.

17. Modèle de données - première cible

Entité	Responsabilité
User	Compte, email, authentification, statut, rôle éventuel.
NutritionProfile	Âge, sexe, taille, poids, activité, objectif, calories/macros, préférences.
Food / Product	Aliment, EAN, marque, valeurs nutritionnelles, source.
Meal / MealItem	Repas, type, date, aliments, quantités / portions.
Household / Member	Foyer, membres, permissions et ressources partagées.
StockItem	Produit, foyer, quantité, unité, emplacement, expiration.
ShoppingList / Item	Articles à acheter, quantité et statut.
DietRule	Contraintes / règles des régimes.
Restaurant / Dish	Restaurant, menu, plat et données nutritionnelles.
Workout / Activity	Séances, activité et métadonnées sportives.
Recommendation	Suggestion, facteurs, type, statut et traçabilité.

18. API et intégrations

- API REST versionnée, par exemple /api/v1.
- JSON cohérent et codes HTTP adaptés.
- Validation Laravel systématique.
- Sanctum pour l'authentification.
- Politiques / autorisations pour isoler les ressources personnelles et de foyer.
- Pagination, filtres et recherche sur les collections importantes.
- Documentation des endpoints et contrats.
- Journalisation des erreurs sans données sensibles.

Open Food Facts

Open Food Facts est la source envisagée en première intention pour la recherche par EAN. Le backend doit mettre en cache les produits utiles dans PostgreSQL et conserver la provenance des données. Les conditions d'utilisation, la qualité des champs et la stratégie de mise à jour devront être vérifiées avant production.

19. Exigences non fonctionnelles

Domaine	Exigence
Sécurité	HTTPS en production, secrets hors dépôt, mots de passe hashés, autorisations serveur, validation des entrées.
Vie privée	Minimisation des données, contrôle du compte et des données, règles de partage du foyer explicites.
Performance	Requêtes PostgreSQL optimisées, cache pertinent, pagination et UX réactive.
Résilience	Gestion des erreurs réseau, états de chargement et reprise après échec.
Maintenabilité	Architecture modulaire, migrations, conventions, tests et documentation.
Compatibilité	Web responsive moderne ; Android prioritaire puis iOS.
Accessibilité	Contrastes, navigation clavier web, libellés, tailles tactiles et information non fondée uniquement sur la couleur.
Observabilité	Logs structurés, suivi d'erreurs et monitoring à sélectionner.

Données de santé et nutrition

Les objectifs, recommandations et estimations nutritionnelles peuvent être sensibles. Le produit devra cadrer les mentions, limites d'usage, consentements et règles de protection des données avant une mise à disposition publique, particulièrement pour les profils spécifiques ou mineurs.

20. Ordre de développement

#	Étape
1	Authentification
2	Gestion de session / routes protégées
3	Profil nutritionnel
4	Dashboard
5	Base aliments
6	Recherche aliments / Open Food Facts
7	Repas
8	Calories et macros journalières
9	Frigo / stock
10	Liaison repas <-> stock
11	Historique
12	Recommandations nutritionnelles simples
13	Restaurants
14	Famille
15	Liste de courses
16	Planning alimentaire
17	Régimes
18	Sport
19	OCR / voix
20	Reconnaissance des assiettes et IA avancée

Priorité actuelle

Authentification -> profil nutritionnel -> dashboard -> aliments -> repas -> frigo.

Le développement immédiat se concentre sur le site Next.js client, tout en conservant une API Laravel compatible avec la future progression de l'application Flutter.

21. Critères d'acceptation du premier socle

- Un utilisateur peut s'inscrire, se connecter, se déconnecter et accéder uniquement aux routes protégées autorisées.
- Il peut créer et modifier son profil nutritionnel.
- Le dashboard affiche des objectifs et totaux cohérents avec les données disponibles.

- La base aliments peut être recherchée et un produit EAN peut être récupéré via le flux local / Open Food Facts.
- Un repas peut contenir plusieurs aliments et les calories/macros sont recalculées correctement.
- L'utilisateur peut gérer son stock frigo/congélateur/placards.
- Les données d'un utilisateur ou foyer ne sont jamais accessibles à un autre sans autorisation.
- Les mêmes endpoints principaux peuvent être consommés par Next.js et Flutter.

22. Tests, environnements et déploiement

Tests

- Tests unitaires des calculs et règles métier.
- Tests API : authentification, validation, autorisations et CRUD.
- Tests d'intégration Next.js <-> Laravel et Flutter <-> Laravel.
- Tests du scan sur appareil Android réel.
- Tests des erreurs : réseau absent, API indisponible, token expiré, source externe incomplète.
- Tests de non-régression sur les calculs nutritionnels et le stock.

Environnements

Environnement	Usage
Local	Serveur local de développement ; Laravel accessible au téléphone sur le réseau ; Next.js en dev ; Flutter en debug.
Staging - futur	Validation avant production avec configuration proche de la production et données de test.
Production - futur	HTTPS, sauvegardes PostgreSQL, secrets sécurisés, monitoring et déploiement reproductible.

Infrastructure locale

Le développement et le stockage sont actuellement prévus sur un **serveur local personnel**. Avant toute ouverture publique, il faudra définir l'exposition réseau, le nom de domaine, HTTPS, les sauvegardes, la restauration, les mises à jour, la disponibilité et la stratégie de sécurité.

23. Points à arbitrer

- Règles exactes de calcul des objectifs calories/macros et bornes de sécurité.
- Gestion des mineurs et profils nécessitant des précautions spécifiques.
- Politique de mise à jour et validation des données Open Food Facts.
- Stratégie de portions et conversions sans pesée.
- Règles de décrémentation automatique du stock et correction utilisateur.
- Permissions détaillées du mode famille.
- Source des données restaurants et géolocalisation.
- Architecture des notifications.
- Mode hors-ligne / cache mobile et web.
- Périmètre et permissions du futur /admin.
- Monétisation éventuelle.

24. Livrables attendus

- Backend Laravel et migrations PostgreSQL.
- Site web Next.js client complet selon les phases.
- Application Flutter Android puis iOS.
- Documentation API et schémas de données.
- Documentation d'installation, configuration et déploiement.
- Tests automatisés et scénarios de recette.
- Documentation des règles nutritionnelles et du moteur de recommandations.
- Futur back-office /admin si son périmètre est validé.

25. Synthèse

NutriApp est conçu comme un **coach nutritionnel et sportif multi-interface**. Son coeur est une API Laravel commune à l'application Flutter et au site Next.js client. Le produit associe suivi nutritionnel, repas, stock du foyer, anti-gaspillage, recommandations, famille, sport et restaurants.

La stratégie de développement est volontairement progressive : consolider l'authentification et le profil, construire le dashboard, la base aliments et les repas, puis relier le stock et les recommandations. L'intelligence avancée intervient ensuite, une fois les données, règles métier et parcours utilisateurs suffisamment fiables.

Interfaces : Next.js constitue l'interface web destinée aux clients, avec une expérience fonctionnelle proche de l'application mobile et adaptée aux ordinateurs. Une interface d'administration distincte pourra être ajoutée ultérieurement via /admin. Aucune interface d'administration Flutter n'est prévue à ce stade.